PostgreSQL VIEWS — Interview Practice Set

5 Combined Easy → Medium → Hard Questions

Focus: CREATE VIEW, querying views, joins, GROUP BY, HAVING, CASE, aggregates, filtering, and updating/dropping views. Designed for interview and coding-round practice. No subqueries, CTEs, or window functions.

Part 1 — Database Setup

Run the following SQL in pgAdmin before attempting the questions.

```
DROP TABLE IF EXISTS enrollments;
DROP TABLE IF EXISTS courses;
DROP TABLE IF EXISTS students;

CREATE TABLE students (
    student_id SERIAL PRIMARY KEY,
    student_name VARCHAR(100) NOT NULL,
    department VARCHAR(50) NOT NULL,
    city VARCHAR(50),
    admission_year INT NOT NULL,
    status VARCHAR(20) DEFAULT 'Active'
        CHECK (status IN ('Active', 'Inactive'))
);

CREATE TABLE courses (
    course_id SERIAL PRIMARY KEY,
    course_name VARCHAR(100) NOT NULL,
    category VARCHAR(50) NOT NULL,
    fee NUMERIC(10,2) NOT NULL CHECK (fee > 0),
    instructor VARCHAR(100) NOT NULL
);

CREATE TABLE enrollments (
    enrollment_id SERIAL PRIMARY KEY,
    student_id INT NOT NULL REFERENCES students(student_id),
    course_id INT NOT NULL REFERENCES courses(course_id),
    enrollment_date DATE NOT NULL,
    marks NUMERIC(5,2) CHECK (marks BETWEEN 0 AND 100),
    enrollment_status VARCHAR(20) NOT NULL
        CHECK (enrollment_status IN ('Enrolled', 'Completed', 'Dropped'))
);
```

Insert Sample Data

```
INSERT INTO students
(student_name, department, city, admission_year, status)
VALUES
('Aarav Sharma', 'Computer', 'Mumbai', 2023, 'Active'),
('Priya Patil', 'Computer', 'Pune', 2022, 'Active'),
('Rahul Verma', 'IT', 'Mumbai', 2023, 'Active'),
('Sneha Joshi', 'Electronics', 'Nashik', 2021, 'Inactive'),
('Aditya Kulkarni', 'Computer', 'Thane', 2022, 'Active'),
('Neha Singh', 'IT', 'Pune', 2024, 'Active'),
('Rohan Deshmukh', 'Mechanical', 'Mumbai', 2022, 'Active'),
('Isha Mehta', 'Computer', 'Navi Mumbai', 2024, 'Active'),
('Karan Shah', 'IT', 'Thane', 2021, 'Inactive'),
('Meera Nair', 'Electronics', 'Pune', 2023, 'Active');

INSERT INTO courses
(course_name, category, fee, instructor)
VALUES
('PostgreSQL Mastery', 'Database', 5000, 'Amit Rao'),
('Python for AI', 'Programming', 6500, 'Neel Joshi'),
('Machine Learning', 'AI/ML', 8000, 'Dr. Kavita Shah'),
('Data Structures', 'Programming', 5500, 'Vikram Patil'),
('Web Development', 'Development', 6000, 'Snehal More'),
('NLP with Transformers', 'AI/ML', 9000, 'Riya Mehta');

INSERT INTO enrollments
```

```
(student_id, course_id, enrollment_date, marks, enrollment_status)
VALUES
(1, 1, '2026-01-10', 92, 'Completed'),
(1, 3, '2026-02-15', 88, 'Completed'),
(1, 6, '2026-03-05', 95, 'Completed'),
(2, 1, '2026-01-12', 78, 'Completed'),
(2, 4, '2026-02-20', 84, 'Completed'),
(3, 2, '2026-01-18', 81, 'Completed'),
(3, 3, '2026-03-01', 73, 'Completed'),
(3, 5, '2026-03-15', NULL, 'Enrolled'),
(4, 4, '2026-01-22', 69, 'Completed'),
(5, 1, '2026-02-01', 91, 'Completed'),
(5, 2, '2026-02-10', 87, 'Completed'),
(5, 6, '2026-03-10', NULL, 'Enrolled'),
(6, 2, '2026-03-02', 76, 'Completed'),
(6, 5, '2026-03-04', 82, 'Completed'),
(7, 4, '2026-02-12', 64, 'Completed'),
(7, 5, '2026-03-12', NULL, 'Dropped'),
(8, 3, '2026-02-05', 89, 'Completed'),
(8, 6, '2026-03-08', 93, 'Completed'),
(9, 1, '2026-01-25', 58, 'Completed'),
(10, 3, '2026-02-18', 71, 'Completed');
```

Part 2 — Practice Questions

Important: Do not look for solutions first. Write the CREATE VIEW statement yourself, then query the view.

Q1 — Easy: Active Student Directory

Create a view named **active_students_view** that contains: student_id, student_name, department, city, and admission_year for only Active students. Then use the view to display all active Computer-department students, ordered by admission_year descending and student_name ascending. Interview focus: basic CREATE VIEW, WHERE, ORDER BY.

Q2 — Easy → Medium: Course Performance View

Create a view named **course_performance_view** using courses and enrollments. For every course, show: course_id, course_name, category, total_enrollments, completed_enrollments, average_completed_marks. Only Completed enrollments should contribute to average_completed_marks. Then display courses whose average_completed_marks is at least 80, ordered from highest average to lowest. Interview focus: JOIN, GROUP BY, COUNT, CASE, AVG, HAVING.

Q3 — Medium: Student Academic Summary

Create a view named **student_academic_summary** using students and enrollments. For every student who has at least one enrollment, show: student_id, student_name, department, total_courses, completed_courses, average_marks. Calculate average_marks using only Completed enrollments. Then query the view to find students who: 1. are in the Computer or IT department, and 2. have completed at least 2 courses, and 3. have an average mark of 80 or above. Interview focus: LEFT JOIN/aggregation logic, conditional aggregation, HAVING, filtering a view.

Q4 — Medium → Hard: Course Difficulty / Performance Classification

Create a view named **course_analysis_view** using courses and enrollments. For each course, show: course_name, category, fee, total_students, average_marks, highest_marks, and a performance_level. Classify performance_level using CASE: • average_marks >= 85 → 'Excellent' • average_marks >= 70 → 'Good' • average_marks >= 50 → 'Average' • otherwise → 'Needs Improvement' Use only Completed enrollments when calculating marks-based metrics. Then query the view to display only AI/ML courses and order them by average_marks descending. Interview focus: CASE inside aggregate queries, conditional aggregation, reusable reporting views.

Q5 — Hard: Interview-Style Training Dashboard View

Create a view named **training_dashboard_view** that combines students, courses, and enrollments. For each course, return: course_id, course_name, category, total_enrolled, total_completed, total_dropped, average_marks, high_scorers (students with marks >= 85), completion_rate. Rules: • total_enrolled = all enrollment records for the course. • total_completed = status 'Completed'. • total_dropped = status 'Dropped'. • average_marks = average of Completed marks only. • high_scorers = Completed students with marks >= 85. • completion_rate = completed / total_enrolled * 100, rounded to 2 decimals. • Avoid division-by-zero errors. Then query the view to return courses where: completion_rate >= 70 AND average_marks >= 75, ordered by completion_rate descending, then average_marks descending. Interview focus: multi-table JOINs, conditional aggregation, CASE, ROUND, NULL handling, arithmetic, view-based reporting.

Part 3 — View Commands You Should Know

```
CREATE OR REPLACE VIEW view_name AS
SELECT ...;

SELECT * FROM view_name;

DROP VIEW view_name;

DROP VIEW IF EXISTS view_name;

-- Inspect views in pgAdmin:
-- Expand Databases → your database → Schemas → public → Views

-- Important interview concepts:
-- 1. A view is a virtual table based on a SELECT query.
```

```
-- 2. It normally stores the query definition, not a separate copy of the result data.
-- 3. Views can simplify complex queries and provide reusable reporting layers.
-- 4. CREATE OR REPLACE VIEW can modify an existing view definition when compatible.
-- 5. A view can be queried like a table.
```

Recommended Practice Order

Q1 → basic view + filtering
Q2 → view + joins + aggregation
Q3 → conditional aggregation + HAVING
Q4 → CASE + reporting logic
Q5 → interview-level combined problem